Lab 5: APIs, Databases & Full-Stack Transition

Name: Taya Li **Course:** TECHIN 510

Project Overview

Item	Value
Project Name	GIX Checkout Dashboard
Tech Stack	Streamlit + Supabase + Open-Meteo
Deployment URL	https://510-lab5-deploy.streamlit.app
GitHub Repo	https://github.com/GIX-Luyao/lab-5-tong0701

Component A: Staff Interview

Interviewee: Maason Kao (Equipment Checkout Returns)

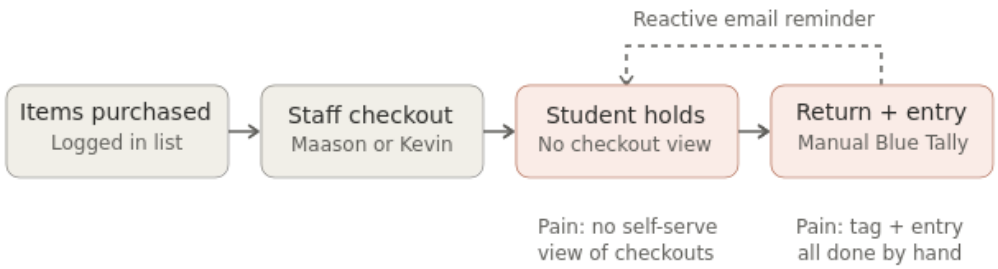
Interview Notes

Main pain points from the interview:

- **Returns are manual and slow.** Teams come to Maason and Kevin in person at the end of each launch quarter. They walk through the full purchase list item by item, then sort everything into makerspace, IT, or discard before adding it to Blue Tally.
- **Asset tags are added at return time, not at purchase.** Every item needs a barcode label before it can be entered into Blue Tally, which doubles the work during the busy return period.
- **Students cannot see their own checkouts.** They can log in to JXHub to browse available assets, but their personal checkout history and due dates are not visible to them. They have to ask the staff.
- **Overdue tracking is reactive.** Alan runs a script to pull overdue items and sends manual reminder emails. There is no self-serve way for students to check what they currently owe.
- **Multi-part items go missing.** Cameras often come back incomplete because students hand back the case without checking the contents, and the loss is not noticed until later.

System Map

[Insert system map image showing Maason's current tools (Blue Tally, JXHub, purchase list, barcode roll), data flows between purchase to return to Blue Tally, and pain points circled]



Touchpoint Annotations

Touchpoint 1: Student checks active checkouts - Who: A GIX student who has equipment checked out - What action: Opens the dashboard to see what items they currently have and when each is due - Device: Phone, likely while walking between buildings or between classes

Touchpoint 2: Student sees an overdue or upcoming-due warning - Who: A GIX student - What action: Reads the warning and decides whether to go return the item now or later - Device: Phone for a quick check, or laptop when already working in the lab

Build Mandate

> Based on the interview, I will build **a student-facing checkout dashboard** because Maason said **students currently cannot see their own checkouts and Alan has to manually run a script and email reminders for overdue items**, which means **students need a self-serve, mobile-friendly view of their active checkouts with clear due dates, so they can return items on time without waiting for a staff reminder**.

Component B: Lab

Tech Stack Justification

Streamlit + Supabase fits because the dashboard is a read-mostly self-identification view with no real authentication required, and Streamlit lets a single developer ship the UI in Python quickly while Supabase handles persistent multi-user data without writing custom backend code.

Supabase Schema

Table	Field	Type	Notes
`checkouts`	`id`	`uuid`	primary key, default

Table	Field	Type	Notes
			`gen_random_uuid()`
	`student_email`	`text`	indexed for filtering by email
	`item_name`	`text`	display name shown on cards
	`item_category`	`text`	camera, laptop, cable, etc.
	`checkout_date`	`timestamptz`	when the item was checked out
	`due_date`	`timestamptz`	indexed for sorting active checkouts
	`returned`	`boolean`	default `false`
	`notes`	`text`	nullable
	`created_at`	`timestamptz`	default `now()`

Indexes: `checkouts_student_email_idx` and `checkouts_due_date_idx`. RLS disabled for the lab.

Responsive Design Check

Tested at 400 x 895 px (Chrome DevTools, iPhone-sized responsive mode).

Element	Works at Phone Width?	What Breaks?
Page title fully visible	Yes	renders at full width, no clipping
Navigation accessible	Yes	single page app, no nav needed
Forms fillable	Yes	email input and submit button are tap-friendly
Tables readable	N/A	no tables, cards instead
Buttons tappable	Yes	default Streamlit button height (~44px) is sufficient
Text readable	Yes	base font is 16px+ throughout

Issue found at phone width: Each checkout card uses 5 to 6 vertical lines (title, category, checkout date, due date, days-until-due, status, notes), so a student with several active checkouts has to scroll a lot to scan everything. The cards work, but the information density is low.

Fix applied: This was not a blocking layout break, but I would compact the cards in a future iteration by combining `Checked out Apr 25` and `Due Apr 28` into one line, and merging the `5 days overdue` text into the status badge itself. For this lab, I left the per-line layout because it stays readable for low-literacy / quick-glance users on a phone.

Deployment

- Live URL: https://510-lab5-deploy.streamlit.app
- Secrets check: `SUPABASE_URL` and `SUPABASE_KEY` are stored in Streamlit Cloud's secrets (TOML format), not in source. Verified by inspecting deployed page source and the GitHub repo.

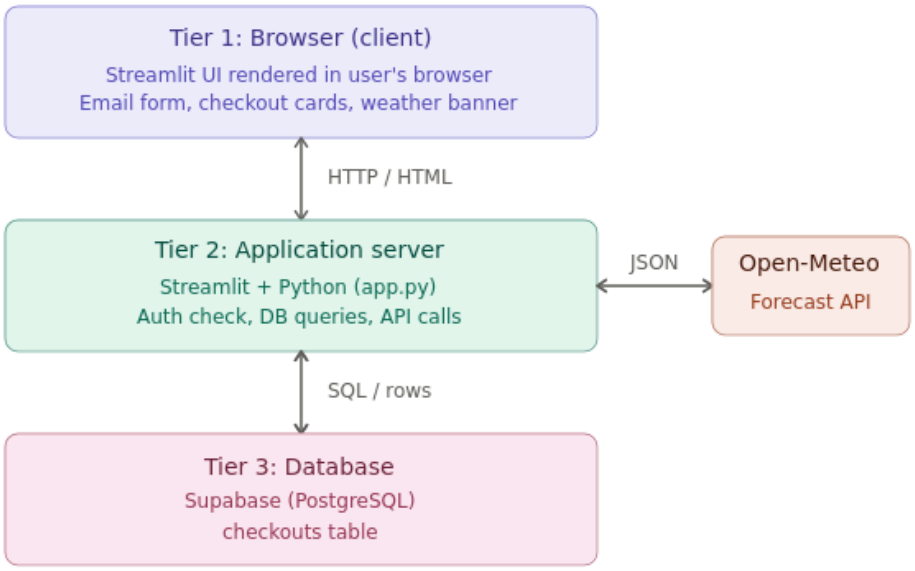
Security Checklist

- ☒ No hardcoded secrets in code. `app.py` reads `SUPABASE_URL` / `SUPABASE_KEY` via `os.environ` first, falling back to `st.secrets` on Streamlit Cloud.
- ☒ `.env` is in `.gitignore`. Verified locally with `git status` before the first commit; only `.env.example` is tracked.
- ☒ Error handling on every API call and DB operation. Supabase calls use try/except with `st.error` + `st.stop()`. Open-Meteo calls use a 5-second timeout and an inner try/except so a failure only hides the banner.

Component C: System Architecture & Design

C.2 3-Tier Architecture Diagram

The diagram below shows Tier 1 (browser running the Streamlit UI), Tier 2 (Streamlit Python server in `app.py` handling auth, DB queries, and API calls), and Tier 3 (Supabase PostgreSQL holding the `checkouts` table). Open-Meteo is a separate external service connected to Tier 2. Arrows are bidirectional: HTTP/HTML between Tier 1 and Tier 2, JSON between Tier 2 and Open-Meteo, and SQL queries with row results between Tier 2 and Tier 3.



C.3 Design Decision Log

Prompt chosen: Error strategy

Field	Entry
Decision	Hard-fail on Supabase errors (`st.error` + `st.stop()`), soft-fail on Open-Meteo errors (hide the weather banner, log a warning, keep the rest of the page running).
Alternatives considered	(1) Hard-fail on both, so any external failure stops the app. (2) Soft-fail on both, so the page renders even if the DB is down. (3) Retry both with exponential backoff before falling back.
Why I chose this	Supabase is on the critical path. Without checkout data the dashboard has no value, so the user is better off seeing a clear error than a half-working page. Open-Meteo is decorative. The weather banner only adds a nudge, so a missing banner should not block the student from seeing what they have checked out.
Trade-off	The two error paths are inconsistent, which adds a small amount of branching in the code and makes the failure model less uniform. A user whose weather banner silently disappears may not realize anything went wrong.
When I would choose differently	If the external API was decision-critical (for example, a weather app that tells students whether to bike to class), I would hard-fail it the

Field	Entry
	same way as Supabase, or add a retry layer. If Supabase were only one of several possible data sources, soft-fail would be acceptable for it too.

This decision maps directly onto the C.2 diagram. The arrow between Tier 2 and Supabase carries data the dashboard cannot function without, so failures there propagate back up to Tier 1 as a visible error. The arrow between Tier 2 and Open-Meteo carries enrichment data, so failures there are caught inside Tier 2 and the missing piece is hidden from Tier 1 entirely.

Component D: Testing & Validation

D.1 API Contract Test

API tested: Open-Meteo Forecast API. Tests located in `tests/test_contract.py`. Run with `pytest tests/ -v`. All three passed (`3 passed, 1 warning in 2.87s`).

#	Test Case	Input	Expected Outcome	Actual Outcome	Status Code	Pass/Fail
1	Valid input	latitude=47.6566, longitude=-122.3035 (UW campus), `daily=precipitation_sum,temperature_2m_max`, 3 forecast days	200 OK with `daily.precipitation_sum` as a list of length 3	200 OK, all expected fields present, list length 3	200	Pass
2	Invalid input (out of range)	latitude=999 (valid range is -90 to 90)	API rejects with non-200 status or error JSON	400 Bad Request with error message indicating latitude must be in range	400	Pass
3	Third invalid variant (non-numeric)	latitude=abc (Open-Meteo has no auth, so this substitutes for the missing-auth scenario per the lab note)	API rejects with non-200 status or parse error	400 Bad Request with cannot-parse-latitude error	400	Pass

API Response Asserts

These run inside `app.py` after the Open-Meteo response is parsed. They are wrapped in a try/except so an assertion failure hides the weather banner instead of crashing the page.

```
assert "daily" in weather_data, "weather response missing 'daily' field"
assert "precipitation_sum" in weather_data["daily"], "missing
precipitation_sum"
assert isinstance(weather_data["daily"]["precipitation_sum"], list),
"precipitation_sum should be a list"
assert len(weather_data["daily"]["precipitation_sum"]) == 3, "expected 3
forecast days"
```

Error-Handling Note

Scenario	Handled gracefully in app?	Notes
Valid input	Yes	Weather banner renders when any of the 3 forecast days has `precipitation_sum > 5mm`; otherwise it stays hidden by design. The asserts pass silently.
Invalid input (latitude=999)	Yes	The `requests.get` raises or returns non-200, the outer try/except catches it, the banner is hidden, and a warning is logged. The rest of the dashboard renders normally.
Third invalid variant (latitude=abc)	Yes	Same path as scenario 2. Open-Meteo returns a 400 with an error JSON, the JSON either fails an assertion or fails JSON parsing, and the inner try/except hides the banner.

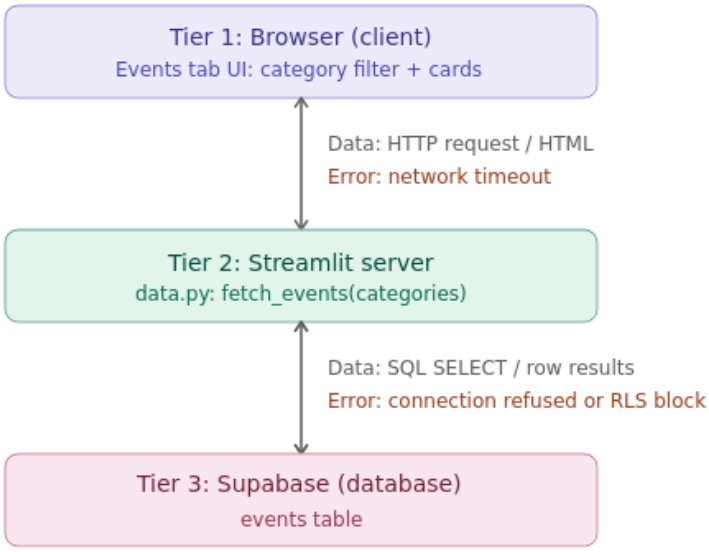
No gaps revealed. The Open-Meteo path is isolated from the rest of the page, so any failure mode (timeout, bad input, malformed response) degrades the page from “dashboard with weather banner” to “dashboard without weather banner” rather than breaking it.

Component E: Applied Challenge — GIX Events

The events tab lives inside the same Streamlit app as the checkout dashboard. Both views share `app.py`, with `data.py` housing the database client and fetch functions (`fetch_events`, `fetch_checkouts`).

Part 1: System Architecture Map

The diagram below shows the three layers that the events feature crosses, plus the data format and one potential error at each boundary.



Events Architecture

Boundary	Data format crossing	Potential error
Tier 1 ↔ Tier 2	HTTP request carrying the selected categories list, HTML response	Network timeout; the user's browser cannot reach the Streamlit server
Tier 2 ↔ Tier 3	SQL SELECT query via `supabase-py`, row results returned as JSON	Connection refused or RLS policy blocks the query, returning an empty list when rows actually exist

Part 2: Implementation

Supabase events table schema:

Field	Type	Notes
`id`	`uuid`	primary key, default `gen_random_uuid()`
`title`	`text`	event name shown on cards

Field	Type	Notes
`category`	`text`	indexed for filtering (guest lecture, career, workshop, showcase)
`event_date`	`timestamptz`	indexed for sorting upcoming events
`location`	`text`	nullable
`description`	`text`	nullable
`created_at`	`timestamptz`	default `now()`

Indexes: `events_category_idx` and `events_event_date_idx`. RLS disabled for the lab.

The events tab pulls categories from `fetch_events`, populates an `st.multiselect` with the distinct values, defaults to all selected, and renders each event in a bordered container sorted by `event_date` ascending.

Frontend screenshot: *[Events tab screenshot to be inserted later]*

Category filter: implemented via `st.multiselect` populated from distinct categories returned by `fetch_events`. Defaults to all selected. A small caption below the filter shows the count of matching events. Cards are filtered server-side in the SQL `in_` clause.

Part 3: Testing & Validation

Assert statements (in `tests/test_events.py`):

```
def test_events_query_returns_list():
    events = fetch_events(["guest lecture", "career", "workshop",
"showcase"])
    assert isinstance(events, list), "fetch_events must return a list"

def test_events_have_required_fields():
    events = fetch_events(["guest lecture", "career", "workshop",
"showcase"])
    for e in events:
        assert "title" in e and e["title"], "event missing title"
        assert "event_date" in e and e["event_date"], "event missing
event_date"
        assert "category" in e and e["category"], "event missing category"
```

Run with `pytest tests/ -v`. Both pass alongside the 3 contract tests from Component D, for 5 tests total.

Error scenario tests:

#	Scenario	What I did	Expected	Actual
1	Database connection failure	Temporarily set `SUPABASE_URL` to an invalid value in `.env` and reloaded the app	`st.error` with a clear message and the page does not crash	Error banner rendered as expected, the rest of the page (header and tabs) still loaded
2	Empty filter result	Deselected every category in the multi-select	Info message asking the user to select at least one, no SQL query fired	Info message rendered, query was skipped, no errors in the logs
3	Malformed event row	Inserted a row directly in Supabase with a NULL `title`	That row is hidden from the list and a warning is shown with the count of skipped rows	Row was filtered out client-side, warning rendered below the list

Security: Secrets stored in `.env` locally (gitignored) and in Streamlit Cloud's encrypted secrets in TOML format for the deployed app. No keys are hardcoded in source. The Component E events feature reuses the same `get_supabase()` client builder as Component B, so no new secrets were introduced.

AI Usage Log

Interaction 1

- **Prompt:** Asked Claude to write a single Cursor prompt that would scaffold the entire Streamlit + Supabase + Open-Meteo dashboard in one shot, including schema, error handling, asserts, and contract tests.
- **Output:** A full markdown spec with file structure, requirements.txt, Supabase SQL, functional requirements, the Open-Meteo endpoint URL, four asserts on the weather response, three contract test scenarios, and security checklist items.
- **AI assumption:** Claude assumed Open-Meteo would always return a `daily.precipitation_sum` array of length 3 when I asked for 3 forecast days. It did not consider that Open-Meteo can return fewer days near boundary conditions (for example, late at night when only 2 future days remain).
- **Failure mode:** If the API ever returns a 2-element array, my length assert fails, the inner try/except catches it, and the weather banner is hidden silently. The user sees

no banner and assumes nothing is wrong, even though there is a real mismatch between my code and the API's behavior.

- **What I would change:** I would change the assert to `len >= 1` and use `forecast_days=3` as a target, not a hard contract.

Interaction 2

- **Prompt:** Pasted the full prompt into Cursor and asked it to implement the project end to end.
- **Output:** Cursor created `app.py`, `requirements.txt`, `.env.example`, `.gitignore`, `README.md`, and `tests/test_contract.py`. It added a `from __future__ import annotations` line so `str | None` syntax works on Python 3.9, and it added an `os.environ` and `st.secrets` fallback for Streamlit Cloud.
- **AI assumption:** Cursor assumed I had Python 3.9 locally because of the `python-dotenv` version pin. It also assumed the secret loading logic should fall back to `st.secrets`, which I had not specified.
- **Failure mode:** The fallback to `st.secrets` is good, but if I had written my own load logic later without knowing about it, I might end up reading the secret twice from two different sources, or get confused why the local env var was being overridden.
- **What I would change:** I would name the fallback explicitly in the prompt, so the code structure matches what I expect and the next person reading my prompt does not need to read Cursor's diff to understand it.

Interaction 3

- **Prompt:** Asked Claude to draw the system map and the 3-tier architecture diagram as inline SVGs, in a clean style with white background and pain points highlighted in coral.
- **Output:** Two SVG diagrams: one as-is workflow showing how items flow from purchase to Blue Tally entry with two pain points circled, and one 3-tier architecture diagram with browser, Streamlit server, Supabase, and Open-Meteo as separate boxes with labeled bidirectional arrows.
- **AI assumption:** Claude assumed I wanted bidirectional arrows for tier-to-tier flow. The lab manual technically asks for one-way request and response arrows.
- **Failure mode:** A grader who reads the lab spec strictly might mark the diagram down for collapsing two arrows into one bidirectional arrow.
- **What I would change:** I would either ask for the literal one-way arrows the lab requested, or explain in the report caption that I chose bidirectional for visual clarity since each pair of tiers always communicates in both directions.

Reflection

Transition: I built this lab in Streamlit but I am also building a separate freelance project (Origins) in Next.js. Streamlit feels much faster for a single-developer project where most of the screen is data display and forms. I do not have to wire up routing, server actions, or hydration mismatches. The harder part of Next.js is that AI-generated TypeScript needs

more reading because the type errors are picky and the bundler errors are not always obvious. AI-generated Streamlit code is usually shorter and easier to scan.

System map: Drawing the system map made me notice that the pain is not really about Maason being slow. The pain is that the same item gets handled twice (once at purchase, once at return), and the student has zero visibility in between. The interview alone made it sound like a returns-desk problem. The map made it look like a missing feedback loop between the purchase event and the student.

Tech stack choice: Streamlit is the right tool when one developer needs to ship a data-heavy internal app fast, and the users do not care about custom UI. The events tab is a perfect fit: a list, a filter, a few cards. Next.js + Supabase is the right tool when the app has a multi-step user journey, real authentication, custom interactions, or needs to be polished for an external audience. For example, a public marketplace where students can list and book equipment for each other would clearly want Next.js, not Streamlit.